

Safe On Route

José Adriel Marcos
Raúl Cristian Dumitru Tanase





ÍNDICE

ÍNDICE	2
AGRADECIMIENTOS	3
José Adriel Marcos Sanchez:	3
Raúl Cristian Dumitru Tanase:	3
INTRODUCCIÓN	4
PLANTEAMIENTO DEL PROBLEMA	5
OBJETIVOS	6
MARCO TEÓRICO	7
Contexto del problema:	7
Hardware:	7
Software:	7
Infraestructura:	8
METODOLOGÍA	9
DESARROLLO DE LA SOLUCIÓN	11
Desarrollo del hardware:	11
Desarrollo de la aplicación web:	12
Despliegue:	13
Problemas encontrados y soluciones:	13
EVALUACIÓN Y RESULTADOS	15
Hardware:	15
Software:	15
Resultados:	15
Limitaciones:	16
Trabajo futuro:	16
CONCLUSIONES	17
BIBLIOGRAFÍA	18
Datos y estadísticas:	18
Tecnologías usadas:	18



AGRADECIMIENTOS

José Adriel Marcos Sanchez:

- Asier y Kevin debido a que formaron parte del proyecto de Safe On Route en Startinnova y en Audicrea y fueron de ayuda en Divulgaciencia sentando las bases del código.
- CPC Salesianos Los Boscos ya que como centro educativo nos ha prestado las instalaciones para poder hacer las pruebas necesarias.
- Roberto Casado por habernos guiado como tutor de TFG y ha sido el profesor con el que hemos podido competir en dichos concursos, ofreciéndonos siempre su ayuda.
- Ignacio López por ser otro profesor con el que competimos en Divulgaciencia y ser de gran ayuda.
- Fundación Caja Rioja, Audicrea y Startinnova por permitirnos participar en sus concursos.

Raúl Cristian Dumitru Tanase:

En primer lugar, agradecer a Roberto y Nacho por el tiempo y la paciencia dedicada a hacerme sentir como un alumno con nombre y apellidos en lugar de hacerme sentir como un número. Y sobre todo por las enseñanzas fuera de los conocimientos técnicos, sois la demostración de que no solo preparáis profesionales sino también personas humanas.

En segundo lugar, me gustaría agradecer al resto de profesores tanto de 1º CFGM de SMR como los de 2º CFGM de SMR el tiempo dedicado a la enseñanza, la paciencia y la vocación real que han demostrado día a día con nosotros en clase.

También me gustaría agradecer a todos mis compañeros de clase sin excepción el tiempo que hemos pasado juntos no solo aprendiendo, sino también socializando y compartiendo recuerdos de dos grandes años de nuestras vidas. Me llevo conmigo amistades que valen fuera del aula.

Por otro lado, un agradecimiento especial a José Luis Tejada, Isaac Heras, Andrés Muñoz y Ángel Requeta por devolverme la motivación para estudiar cuando llegué a Salesianos Los Boscos desilusionado y con un futuro incierto.

Por último, agradecer en mi último año como alumno de Salesianos Los Boscos las facilidades que se han proporcionado y sobre todo el increíble profesorado que sin duda, cumple con las expectativas de lo que verdaderamente es la educación de calidad.

The only way to do great work is to love what you do.



INTRODUCCIÓN

Los accidentes de tráfico siguen siendo una de las principales causas de muerte evitable en España y Europa. En 2024, 1.154 personas fallecieron en carreras españolas y cerca de 19.800 lo hicieron en el conjunto de la Unión Europea. Muchas de estas muertes podrían haberse evitado con una respuesta más rápida y precisa.

Este trabajo de final de grado aborda este problema mediante el diseño y desarrollo de Safe On Route, un sistema de detección automática de accidentes de tráfico en tiempo real. El objetivo del mismo es reducir el tiempo de activación de los servicios de emergencia aprovechando la hora de oro.

La hora de oro es un periodo crítico de 60 minutos tras un accidente en el que la atención médica puede marcar la diferencia entre la vida y la muerte.

Para lograrlo, se ha desarrollado un dispositivo que navega a bordo del vehículo integrando múltiples sensores; acelerómetro, GPS, sensor de inclinación, células de carga y puerto OBDII, entre otros. Este dispositivo es capaz de detectar y caracterizar un accidente automáticamente. Además, el dispositivo transmite los datos a través de una aplicación web a un operador, dicho operador actúa como enlace con los servicios de emergencia, enviando el recurso más adecuado según la gravedad o tipología del accidente.

Como resultado, en la trayectoria de Safe On Route se han obtenido dos prototipos funcionales aunque en el presente trabajo se hace uso solamente de uno al no tener la posibilidad de utilizar un coche en tiempo real durante el desarrollo del mismo. A pesar de ello, con el prototipo actual se demuestra la viabilidad técnica del sistema con un coste accesible y con una arquitectura escalable hacia otros tipos de vehículos.



PLANTEAMIENTO DEL PROBLEMA

Los accidentes de tráfico constituyen una de las principales causas de muerte evitable tanto en España como en el conjunto de la Unión Europea. En 2024, 1.154 personas perdieron la vida en carreteras españolas, mientras que la cifra europea ascendió a 19.800 fallecidos. Según estimaciones de ERSO, hasta un 25% de las víctimas mortales podrían haber sobrevivido si los servicios de emergencia hubieran actuado dentro de la denominada hora de oro, el período crítico de 60 minutos tras un accidente durante el cual la atención médica resulta determinante para la supervivencia.

El problema central no es únicamente el hecho de que sucedan los accidentes, sino el retraso en la activación de los servicios de emergencia. En la mayoría de accidentes, la alerta a los servicios de emergencia depende de un testigo o de que el propio accidentado realice la llamada (Lo cual no es posible en muchos casos). Esta estructura a pesar de ser la más común, presenta fallos evidentes; en accidentes nocturnos, en zonas de escasa visibilidad o en vías poco transitadas, puede transcurrir un tiempo considerable hasta que se produce la llamada. Además, la información que se le suele transmitir a los servicios de emergencia se habitúa a ser escasa o errónea. Esto último dificulta a los servicios de emergencia la posibilidad de dimensionar correctamente los recursos necesarios para atender a las víctimas al llegar a la zona del siniestro.

A todo esto se suma que los sistemas de detección automática de accidentes existentes en el mercado presentan limitaciones relevantes. O bien están integrados solamente en vehículos de alta gama, o bien se basan en sensores de uso general como los acelerómetros de un smartphone (que suele tener una gran tasa de falsos positivos). Ninguna de estas soluciones caracteriza realmente un accidente como tal (tipología, gravedad estimada, número de ocupantes, estado del vehículo etc).

Por lo tanto, existe una necesidad real y no resuelta, un sistema de bajo coste capaz de detectar automáticamente un accidente, que además, lo caracterice con precisión mediante el uso de múltiples sensores y que transmita dicha información a un operador que pueda redirigir los recursos más acertados. Safe On Route surge como respuesta técnica a esta necesidad.



OBJETIVOS

El objetivo real de Safe On Route es reducir el tiempo de activación de los servicios de emergencia tras un accidente de tráfico, actuando en la hora de oro para maximizar las posibilidades de supervivencia de las víctimas.

Para alcanzar el objetivo principal, se plantean los siguientes objetivos específicos:

- Objetivos técnicos:
 - Diseñar e implementar un dispositivo capaz de detectar automáticamente un accidente de tráfico mediante la fusión de datos de múltiples sensores; acelerómetro, sensor de inclinación, células de carga, GPS, etc...
 - Desarrollar un algoritmo de clasificación que determine la gravedad y tipología del accidente a partir de los datos recogidos.
 - Implementar un sistema de transmisión de datos en tiempo real entre el dispositivo y un servidor centralizado.
- Objetivos de viabilidad del proyecto:
 - Demostrar la viabilidad técnica del sistema con un prototipo funcional desarrollado con componentes accesibles y de bajo coste (Arduino).
 - De forma que cualquier usuario lo pueda adoptar en su vehículo.
 - Validar que la arquitectura del prototipo es escalable hacia otro tipo de vehículos más allá de los coches.
- Objetivos a medio plazo:
 - Sentar las bases para desarrollar una aplicación móvil/web complementaria que permita notificar a familiares y almacenar datos médicos relevantes para los servicios de emergencia.



MARCO TEÓRICO

Contexto del problema:

Realmente, la hora de oro es un concepto médico que establece que las víctimas de un traumatismo grave tienen muchas más probabilidades de sobrevivir si reciben atención médica en los primeros 60 minutos tras el accidente. Según estimaciones del ERSO (European Road Safety Observatory), hasta un 25% de las víctimas mortales podrían haber sobrevivido con una respuesta más rápida.

Desde abril de 2018 la Unión Europea obliga a que todos los vehículos incorporen un sistema de eCall, que realiza una llamada automática en caso de accidente grave. Sin embargo, esta normativa solo aplica a vehículos fabricados a partir de 2018 y no a millones de vehículos ya existentes y en circulación...

Hardware:

Arduino es una plataforma Open Source de creación de electrónica. Sus estándares se caracterizan por tener un hardware y un firmware flexible y fácil de utilizar para los desarrolladores. Estos son los principales motivos por los cuales fue elegido para el primer prototipo del proyecto. Se necesitaba algo fácil de desarrollar en primera instancia con un coste mínimo. En nuestro caso se eligió una placa Arduino UNO R3 aunque no hubiera sido incorrecto utilizar una placa Arduino UNO R4 dado que integra nativamente un módulo WiFi. A grandes rasgos se puede mencionar que todos los sensores elegidos se guiaban bajo la misma filosofía que la elección de la placa base, fiabilidad, facilidad y coste mínimo.

Otro de los items más importantes de entender a nivel hardware, es el OBD-II. El OBD-II es un puerto de diagnóstico disponible en todos los vehículos desde 1996. Permite leer datos del sistema electrónico del vehículo; velocidad, RPM, temperatura, combustible, etc. En nuestro proyecto, el puerto OBD-II está presente en ediciones pasadas que fueron presentadas a concursos como Divulgaciencia o StartInnova. Sin embargo, actualmente no operamos el puerto pero en versiones futuras no se descarta volver a utilizarlo para rescatar datos con menos infraestructura.

En nuestro caso, en versiones pasadas de Safe On Route se usaban dos formas de enviar los datos. La primera fue mediante el uso de ThingsBoard, una plataforma que integra su propio sistema de API para recibir y mostrar datos gráficamente. La segunda fue mediante un pipeline de Kafka. Ambas formas fueron descartadas para dar paso a la más eficiente. El envío con peticiones HTTP a la propia API de la aplicación web de Safe On Route. Este envío se realiza mediante el endpoint `/api/realData/create` y precisa de API Key en el header de la petición para autenticarse.

Software:

Lo más destacable del software además de la propia programación del firmware de Arduino, es la aplicación web de Safe On Route. Con el desarrollo de la aplicación web se quería intentar ser lo más escalable posible ¿Qué sucede si el día de mañana aumenta el tráfico y hay que migrar tecnologías como el ORM? ¿Qué sucede si el PI se convierte en un proyecto empresarial serio y precisa de una mejor estructuración? Todas esas preguntas las responden tanto la arquitectura hexagonal como React. Se escogió la arquitectura hexagonal debido a que aporta desacoplamiento, escalabilidad y facilidad para futuros desarrolladores.



Además, React, aporta la descomposición del frontend en componentes reutilizables que pueden encajar y desencajar como piezas de un puzle sin dificultad alguna. Además, aunque el proyecto no obtenga todas las medidas que un proyecto empresarial debería tener, sí es cierto que se optó por medidas de seguridad introductorias como; middlewares de autenticación, validación en los body HTTP, argon2 para el hashing de contraseñas, cookies httpOnly para el token de inicio de sesión, rate limiters, ORM para proteger contra inyecciones SQL al parametrizar sentencias, entre otras más.

Infraestructura:

Para el apartado DevOps se escogió una solución al mismo nivel que el propio proyecto pero igual de escalable que la arquitectura hexagonal, Amazon Lightsail. Lightsail ofrece instancias baratas con pre-instalación de node en la VPS. Además, la instancia cuenta con un reverse proxy, configurado manualmente con Nginx, que redirige el tráfico del puerto 80 al 3000 que es donde corre la aplicación web. Esto permite que el usuario no tenga que introducir al puerto al que quiere acceder de esta forma; dominio.es:3000 sino que queda como dominio.es.

Por otra parte, si la aplicación se ejecuta directamente en node y se cierra la terminal SSH, el proceso morirá con la terminal. Por ende, se utiliza un gestor de procesos que mantiene la app viva en segundo plano, en caso de caerse se autoreinicia y arranca el servidor.

Para usar más conocimientos obtenidos en el segundo curso de SMR, se decidió utilizar Certbot para obtener un certificado SSL gratuito de Let's Encrypt para cifrar las comunicaciones y obtener así HTTPS en lugar de HTTP. El reverse proxy se encarga automáticamente de redirigir el tráfico del puerto 80 al 443.

Por último, es importante destacar que la base de datos se aloja de forma externa en un servidor de Pienza Solutions. Esta decisión se vio un poco forzada por dos motivos. El primer motivo es que dicha base de datos ya existía, crear otra aunque fuera en local no hubiera tenido mucho sentido. El segundo motivo, es que la instancia de Lightsail no hubiera sido capaz de soportar el tráfico de una base de datos local sin optar por una más cara que la instancia actual.



METODOLOGÍA

En primera instancia cabe mencionar que el trabajo realizado para identificar el problema se ratifica más en lo teórico que en lo práctico. Por ende, previo a todo el desarrollo del hardware y software se realizó un pequeño trabajo de investigación donde se analizaron tanto las estadísticas relacionadas con los siniestros y fallecidos en carretera como el análisis de otras corporaciones que trataran de solucionar de una forma parecida el mismo problema.

El análisis de siniestros y fallecidos fue un trabajo relativamente fácil dada la cantidad de información que proveen instituciones públicas como la DGT en España o la propia Unión Europea. Sin embargo, el research realizado respecto a la competencia merece la pena explicarlo más detalladamente.

1. Dispositivos de consumo:

- Es una clasificación referente a dispositivos no embebidos en los propios vehículos, sino que son los smartphones o wearables quienes adoptan esa funcionalidad. Las tech que más han conseguido masterizar esto son Apple con su ecosistema de Apple Watch + iPhone y Alphabet con el Google Pixel 4a. Estos dispositivos detectan impactos frontales, laterales, traseros y vuelcos además de alertar automáticamente a los servicios de emergencia si no hay una respuesta antes de los 10 segundos después de un posible impacto. Una característica que resulta interesante es que notifica a su vez a todos los contactos de emergencia de la persona que ha resultado tener el presunto accidente. Todo lo mencionable referente a estos dispositivos es importante remarcarlo con un presunto delante dado que el hecho de ser wearables provoca que existan muchos falsos positivos cuando se realizan actividades de alto impacto.

2. Hardware embebido:

- En estas empresas es importante destacar **quién** es target donde aplica la tecnología. Con los dispositivos de consumo se identificaba sin necesidad de mención el hecho de que el target era el consumidor final. Sin embargo, el hardware embebido tiene como target el fabricante para que este lo implemente en sus productos. Algunas empresas que destacan son Valeo (Fabricante para Renault) o Mobileye.

Dado este research, podemos sacar en conclusión que no existe un término medio real en el mercado sino que se encuentra bastante dividido. Ese es el punto donde se encuentra Safe On Route, una solución tanto para usuarios finales como para fabricantes en sí. Esto se debe a que es un hardware fácil de montar en un vehículo ya fabricado y fácil de implementar en vehículos en estado de fabricación.



En segunda instancia, una vez con la investigación realizada se podía plantear el stack de tecnologías válido para la magnitud del proyecto. Safe On Route es un proyecto basado en hardware estrictamente de Arduino, esto obliga al proyecto a ser programado en C++. Sin embargo, otras partes del proyecto, como la recepción remota o no remota de los datos, depende de otras tecnologías.

Stack tecnológico utilizado organizado por secciones:

1. IDEs:
 - Visual Studio Code.
 - Arduino IDE.
2. Hardware:
 - Arduino UNO R3.
 - ADXL345.
 - HX711.
 - ESP8266.
 - Potenciometro.
 - Finales de carrera.
3. Frontend:
 - React 19.
 - Vite.
 - Axios.
4. Backend:
 - Node.js.
 - Express 5.
 - Sequelize.
 - MySQL.
5. Hosting:
 - AWS Lightsail.
 - Nginx.
 - PM2.
 - Certbot.

Como dispositivo extra usado fuera del stack mencionado anteriormente, se usó un OBD Reader para poder leer datos provenientes directamente de un vehículo y tratar de ahorrar en sensores y desarrollo de software. Todo esto fue posible gracias al uso de ingeniería inversa, con el reader conectado se iban realizando acciones para detectar cual era el trigger que provocaba el evento y así poder usarlo a nuestro favor en el display de datos en la página web posteriormente.

La maqueta inicial proviene del concurso divulgaciencia 2025. Con un asiento de Citroen C3 y una estructura metálica fuimos capaces de ensamblar diferentes de los sensores mencionados y paralelamente leer los datos con el OBD Reader de un vehículo real en movimiento para demostrar la capacidad real del proyecto en el poco tiempo que se tuvo de desarrollo. Por parte de la maqueta pudimos sacar; el estado del cinturón, los kg de fuerza con los cuales se golpeaba el parachoques delantero, de forma booleana la existencia de un pasajero, la posición tridimensional del vehículo y el porcentaje de frenado con la simulación de un pedal. Por parte del vehículo real en movimiento se consiguieron sacar datos de aceleración, posición GPS, volumen de combustible, temperatura etc.



Todos estos datos se enviaban mediante módulos wifi y eran recibidos por una API que hacía el display de los datos en una página web. En caso de inconvenientes inesperados se podía recibir los datos mediante un cable a un ordenador portátil y mostrarlos por consola.

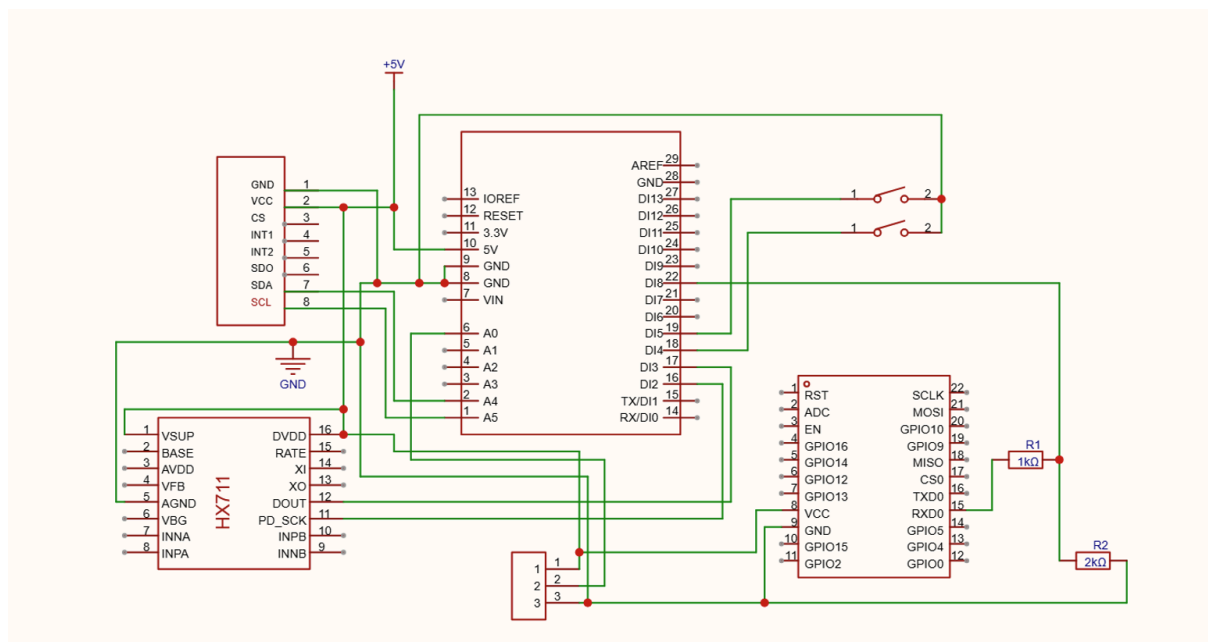
DESARROLLO DE LA SOLUCIÓN

Desarrollo del hardware:

El módulo físico se construyó sobre un Arduino UNO R3 que centraliza la lectura de sensores y la transmisión WiFi. Los componentes de los cuales se conforma son:

- **ADXL345:** acelerómetro de 3 ejes configurado a $\pm 2g$, mide aceleración y permite calcular pitch y roll del vehículo.
- **HX711 + célula de carga:** detector principal de impactos, soporta hasta 20kg con calibración persistente gracias a EEPROM.
- **Potenciómetro:** Simula el freno del vehículo, tiene una lectura analógica 165-868 convertida a 0-100% con un filtro de ruido electrónico.
- **Finales de carrera:** Detectan cinturón y la existencia de un ocupante de forma booleana.
- **ESP8266:** módulo WiFi que envía datos por HTTP. Necesita un divisor de tensión porque opera a 3.3V contra los 5V del arduino.

El firmware funciona como una máquina de estados con el siguiente flujo: conecta el WiFi → hace login con API Key → envía telemetría cada 4 segundos al endpoint sanitizando datos que sean NaN. El esquema de conexionado para replicar el cableado del hardware es el siguiente:



Para más detalle sobre el esquema de conexión, firmware y protocolo de comunicación consultar el anexo 1.



Desarrollo de la aplicación web:

1. **Arquitectura:** Para el desarrollo como se ha mencionado anteriormente se utilizó la arquitectura hexagonal con tres capas; domain, application e infraestructure. El motivo es su gran escalabilidad y capacidad de desacoplamiento.
2. **Modelo de datos:** La aplicación web cuenta con 5 tablas: user, vehicles, emulatedData, realData y alerts. Es importante destacar la relación polimórfica entre alerts.crash en base a alerts.type. En caso de ser type = "emulated", el campo alerts.crash apuntará a emulatedData.emulatedDataId. En caso de ser type = "real", lo hará a realData.realDataId. Sin embargo, existen relaciones 1:n más sencillas como: un usuario tiene muchos vehículos y un vehículo tiene muchos datos y alertas.
3. **API REST:** Toda la API de Safe On Route se encuentra bajo el prefijo /api/ que se extiende por cada modelo de la base de datos. Es importante destacar que a excepción de /api/realData/create, todos usan autenticación mediante JWT. Esto se debe a que dicho endpoint es utilizado por un dispositivo IoT y no por un usuario, por lo que autentica mediante API Key.
4. **Sistema de autenticación:** Actualmente el registro de usuarios existe solamente mediante peticiones HTTP autenticadas en Postman, sin embargo se trata de una funcionalidad futura para la aplicación web. El login a su vez genera un JWT guardado en una cookie httpOnly (no localStorage por seguridad XSS). El middleware "authMiddleware" verifica el token en cada petición que realiza el usuario. Por otro lado, el middleware rateLimiter limita las peticiones por IP para evitar ataques de fuerza bruta en el login.
5. **Sistema de alertas:** Las alertas siempre son generadas de manera automática, sea por un script o por la propia ejecución del flujo del sistema. Existen dos tipos de flujo que se diferencian mediante el campo type en la tabla alerts tal y como se ha mencionado anteriormente. El flujo emulado consta de 12 sensores de impacto distribuidos por los 4 lados del vehículo y otros sensores que prestan información de cómo ha sido el accidente. El flujo real recrea a menor nivel el emulado con la célula de carga.
6. **Frontend:** La aplicación web incluye páginas informativas sobre el funcionamiento del producto, así como páginas funcionales. Entre estas últimas destacan el dashboard donde se reciben las alertas y se gestionan los vehículos y la página de documentación donde se puede leer tanto esta memoria como los diferentes anexos del PI.

Para más detalle del desarrollo de la aplicación web consultar el anexo 2.



Despliegue:

1. **Servicios cloud:** Como servicio de despliegue cloud se utiliza AWS Lightsail con una máquina Debian de 1GB de RAM y 2Vcores. Se eligió este servicio dado que el volumen de tráfico es bajo y su coste es reducido. En caso de precisar una máquina más potente la decisión correcta sería migrar la aplicación a un EC2 con Amazon Linux.
2. **Base de datos externa:** La base de datos en este caso se encuentra externalizada en un servidor de Piensa Solutions principalmente por dos motivos: su reutilización, ya que el servidor ya existía previamente y porque la instancia actual de Lightsail no sería capaz de soportar la carga de incluir también la base de datos..
3. **Flujo de arranque y recepción del código:** En primer lugar, se realiza un git pull en la VPS para recibir las nuevas funcionalidades o correcciones que vengan del trabajo en local. A continuación se vuelve a construir el frontend y por último se reinicia el proceso de PM2 para que actualice la aplicación web. PM2 es clave para mantener el backend encendido las 24h del día y para gestionar posibles crashes que pudieran parar la aplicación.
4. **HTTPS:** Se obtuvo un certificado SSL con Certbot y Let's Encrypt. Gracias a Nginx todo el tráfico que aun accede a la aplicación web mediante el puerto 80 es redirigido al puerto 443. Además Nginx brinda el reverse proxy con una configuración sencilla proporcionando así una navegación más limpia al no requerir que el usuario introduzca el puerto en la URL.
5. **Dominio:** Proporcionado por el centro educativo, se obtuvo un dominio, igualmente alojado en Piensa Solutions apuntando con un DNS type A a la IP pública de la instancia de Lightsail.

Para más detalle del despliegue de la aplicación web consultar el anexo 3.

Problemas encontrados y soluciones:

- **Hardware:** A nivel hardware el mayor de los problemas ha sido el envío de datos de telemetría desde el Arduino al servidor. Esto se debía a que las peticiones HTTP llegaban con un payload corrupto o incompleto. El origen del problema estaba en la comunicación entre el Arduino y el ESP8266. Dicha comunicación se realiza mediante SoftwareSerial a 9600 baudios (aproximadamente 960 caracteres por segundo). Es importante destacar que como SoftwareSerial emula un puerto serial por software, si la comunicación se hiciera a mayores velocidades, se volvería inestable. A su vez, el ESP8266 dispone de un buffer de 64 bytes donde se almacenan los datos temporalmente antes de procesarlos y enviarlos por WiFi. Por lo tanto, al intentar transmitir el payload completo de una sola vez, el buffer se llenaba y se perdían bytes resultando en transmisiones corruptas o incompletas. La solución consistió en fragmentar el envío en bloques de 64 bytes con pausas de 10 ms entre cada bloque. Esto permitía al módulo procesar cada fragmento antes de recibir el siguiente y transmitir el payload completo.



- **Software:** Durante el desarrollo de la aplicación web existieron gran cantidad de problemas. Los más destacables y de los cuales más aprendizaje se pudo obtener fueron:
 1. **Case sensitive:** El desarrollo en local fue realizado en Windows, naturalmente Windows no es case sensitive por ende los imports de dependencias podían poseer mayúsculas aunque luego el nombre del archivo realmente no tuviera dichas mayúsculas. Al realizar el despliegue en AWS que utiliza Linux, esto fue un problema dado que Linux sí es case sensitive y los imports erróneos tuvieron que ser cambiados.
 2. **Puerto 80 ocupado:** Al tratar de configurar Nginx se descubrió que la versión de Node pre-instalada por Lightsail ya se encontraba con un proceso abierto en dicho puerto. Esto imposibilitaba la configuración, para solucionarlo se forzó el cierre de cualquier proceso existente en el puerto 80 dando así paso a Nginx.
 3. **Rama equivocada en la VPS:** En el repositorio de github se indicó de manera intencionada que la rama principal debía ser “develop” y no “main” para que los PR no llegaran directamente a producción sino que existiera un filtro anterior que no rompiera la aplicación web activa. Esto provocó que la VPS inicializase todo desde la rama “develop” lo que consideramos un problema dado que algunas correcciones o *chores* se realizaban directamente sobre la rama “main” para ahorrar pasos y la creación de ramas demasiado simples.
 4. **.env con espacios:** Al final del desarrollo y del despliegue el login daba status 500 porque las variables del archivo .env poseían espacios alrededor del “=”. Fue un error muy difícil de detectar dado que el archivo .env aparentaba estar correctamente configurado y el mensaje de error no indicaba directamente la causa del problema.



EVALUACIÓN Y RESULTADOS

Para empezar, hay que destacar que todas las pruebas realizadas fueron manuales. De forma profesional se es consciente de que no es la práctica más adecuada ya que lo ideal hubiera sido implementar test automatizados.

Hardware:

Una vez montados todos los sensores y programado el firmware se fueron probando los sensores uno a uno, girando, golpeando, agitando y moviéndolos para provocar datos e interpretar el cómo se generan. En caso de registrar algún evento que no encajase con la acción aplicada sobre el sensor se buscaba la manera de resolver el error ya sea mirando documentación referente al sensor, debuggeando el código del firmware o incluso reemplazando el propio sensor por otro idéntico.

Software:

En este caso las pruebas eran bastante más sencillas y sobre todo, los errores mucho más explicativos que con el firmware del hardware. Simplemente se fue probando cada endpoint manualmente con postman y posteriormente se repetía la acción con el componente del frontend. Las pruebas del sistema de alertas fueron más complejas, dado que requerían programar scripts en JavaScript que simularán payloads similares a los que recibiría el dashboard en un entorno real.

Resultados:

A continuación se evalúa el grado de cumplimiento de los objetivos propuestos:

- Objetivos técnicos:
 - Dispositivo capaz de detectar un accidente con múltiples sensores: **CUMPLIDO**. Se diseñó y construyó un prototipo basado en Arduino UNO que integra un acelerómetro, una célula de carga con amplificador HX711, un potenciómetro y dos finales de carrera. Dicho dispositivo puede detectar impactos, inclinación y presión, transmitiendo los datos de forma autónoma.
 - Algoritmo de clasificación por gravedad y tipología: **CUMPLIDO**. Se diseñó un algoritmo que a partir del payload transmitido por el dispositivo físico o por scripts emulados es capaz de clasificar la gravedad del accidente y de qué parte del vehículo proviene.
 - Transmisión de datos en tiempo real al servidor: **CUMPLIDO**. Se diseñó e implementó un sistema de API REST capaz de recibir los datos transmitidos por el módulo físico para su interpretación.
- Objetivos de viabilidad:
 - Prototipo funcional con componentes de bajo coste: **CUMPLIDO**. Los componentes utilizados no suelen superar los 10€ por unidad lo que provee de un módulo físico muy económico.
 - Cualquier usuario puede adoptar esta tecnología: **PARCIALMENTE CUMPLIDO** dado que requiere ciertos conocimientos técnicos. A pesar de requerir dichos conocimientos técnicos se proporcionan manuales para montarlo con relativa facilidad.
 - Arquitectura escalable a otros vehículos: **CUMPLIDO**. La arquitectura hexagonal permite escalar la aplicación web con mucha facilidad. A su vez, Arduino es una tecnología de iniciación para escalar a un hardware más profesional.



- Objetivos a medio plazo:
 - Bases para una app móvil/web con notificaciones a familiares y datos médicos: **PARCIALMENTE CUMPLIDO** a falta de posibles implementaciones futuras sobre datos médicos y plan familiar. A pesar de ello, la aplicación web ya recibe, procesa y visualiza datos en tiempo real, sentando las bases para futuras funcionalidades.

Limitaciones:

Es una realidad que al tratarse de un PI existen claras limitaciones sobre todo a nivel presupuestario que no permiten desarrollar una tecnología más avanzada o de mayor calidad. A pesar de ello, cabe mencionar también que no todas las limitaciones provienen de agentes externos sino también de los propios autores, ya sea por su nivel de conocimiento, dedicación o tiempo disponible para dedicarle al PI. A continuación se presenta una lista de limitaciones conocidas:

- El prototipo físico actual solo recrea los sensores a pequeña escala, no se ha podido escalar a un prototipo con mayor cantidad de sensores y tampoco ha podido ser probado en un vehículo real en movimiento.
- El registro de usuarios solamente puede ser realizado mediante Postman, no existe un formulario de registro en la web.
- No existen tests automatizados.
- El GPS no está integrado en el prototipo actual.

Trabajo futuro:

A pesar de las limitaciones ya conocidas sí existe la intención de continuar el proyecto y desarrollar nuevas funcionalidades:

- Aplicación móvil en React Native para la notificación a familiares.
- Integración de datos médicos del conductor.
- Añadir un GPS real al módulo.
- Registro de usuarios desde la propia web.
- Test automatizados con Jest.
- Migrar a un hardware más profesional.
- Integración con OBD-II real del vehículo.



CONCLUSIONES

Se ha construido un sistema completo de extremo a extremo, desde el dispositivo físico con sensores que detectan el accidente, pasando por la transmisión de datos en tiempo real hasta el dashboard web donde un operador visualiza y gestiona las alertas. Todo esto ha sido realizado con componentes de bajo coste demostrando que no es necesaria una gran inversión para desarrollar un sistema de detección automática de accidentes. Además el sistema está desplegado en un entorno real de producción accesible desde internet con dominio propio y HTTPS, no se trata solo de un prototipo local.

A nivel técnico se ha aprendido a desarrollar un proyecto fullstack con arquitectura hexagonal y con un diseño propio de API REST, autenticación con JWT, trabajo con sensores y firmware en Arduino, despliegue en la nube con AWS, configuración de servidores Linux, Nginx, PM2, certificados SSL, etc. A nivel profesional, se ha aprendido a gestionar un proyecto desde el principio hasta el final, solventando problemas espontáneos tanto en desarrollo como en producción y a documentar todo el trabajo realizado.

1.154 han sido los fallecidos en carreteras españolas en 2024, como se ha visto previamente, el 25% podrían haber sobrevivido si la atención sanitaria hubiera sido más veloz. Este proyecto demuestra que con tecnología accesible se pueden sentar las bases para reducir esas cifras. Cualquier solución que ayude a cumplir con los objetivos propuestos es motivo más que suficiente para continuar con el desarrollo.



BIBLIOGRAFÍA

Datos y estadísticas:

- [Fallecidos en carreteras Españolas.](#)
- [Fallecidos en carreteras de la UE.](#)
- [Hora de oro.](#)
- [eCall UE.](#)

Tecnologías usadas:

- [React](#), [Express](#), [Sequelize](#), [Vite](#).
- [Arduino](#), [ADXL345](#), [HX711](#), [ESP8266](#).
- [AWS Lightsail](#), [Nginx](#), [PM2](#), [Certbot](#).
- [MySQL](#), [JWT](#).
- [Arquitectura hexagonal.](#)